The PSMFlows Pipeline, in Plain RL Terms

Companion to `main.tex` — August 2026

1 The problem in one paragraph

We are given a fixed dataset $\mathcal{D} = \{(s_i, a_i, s'_i)\}$ of transitions collected by some unknown behavior policy, *with no rewards*. Later, someone hands us a reward function, and we must produce a good policy *without any further environment interaction* (zero-shot offline RL). Methods like FB and PSM solve this by learning successor measures — "if I act like policy π from state s, which states will I visit?" — for a whole family of policies at once. Their weakness: during training they evaluate candidate *actions* that may never appear in the data. The value estimates for such out-of-distribution (OOD) actions are unconstrained, the argmax hunts for exactly those over-estimated actions, and the resulting policy is garbage precisely where the data is thin. PSMFlows removes this failure mode by construction: **every action the method ever considers is produced by a generative model of the data itself.**

2 The key idea: index policies by flow noise

Train a conditional flow $G_\theta(s, u)$ that maps a noise vector $u \sim \mathcal{N}(0, I_{d_a})$ to an action that looks like the dataset's actions at state s (behavior cloning with a flow, exactly the BC part of FQL). Because the noise dimension equals the action dimension, $G_\theta(s, \cdot)$ is an invertible map: every noise vector decodes to a data-like action, and every action has a unique noise vector that produces it.

Now the trick: *hold the noise fixed.* The deterministic policy $\pi_u(s) = G_\theta(s, u)$ takes, at every state, "the kind of action the data takes at s, in direction u." Different u's give different, reproducible behaviors (our D2 diagnostic measures exactly this), so the noise vector u becomes a *policy index*. Optimizing over $u \in \mathbb{R}^{d_a}$ replaces optimizing over raw actions — and any u of typical norm decodes to an action the data supports. The hard constraint "stay on the data manifold" becomes the easy constraint "keep $\|u\|$ typical."

3 The three training stages

3.1 Stage A — learn the behavior flow (reward-free BC)

Standard conditional flow matching: sample a dataset pair (s, a), a noise u_0, a time t, and regress the velocity network $v_\theta(s, (1-t)u_0 + ta, t)$ onto $(a - u_0)$. Integrating v_θ from u over $t \in [0, 1]$ gives the decoder $G_\theta(s, u)$; a one-step student network is distilled from the integrated flow for fast acting. In the code this is `agent=fql agent.bc_only=true` (the critic is never trained; rewards are never read). **The flow is then frozen forever.** Freezing matters: the

policy family that the next stages build on is stationary, unlike FB's actor which moves during training.

3.2 Stage B — invert the flow over the dataset (preimages)

Stage C will need to know, for every dataset transition, *which latent the behavior "used"*: the u with $G_\theta(s,u) \approx a$. Since $G_\theta(s,\cdot)$ is invertible, run its ODE backwards from a to get the point preimage $u^\star = E_\theta(s,a)$. Because a whole basin of latents decodes to nearly the same action, we also fit a distribution over that basin, the relaxed posterior

$$q_\alpha(u \mid s,a) \;\propto\; \mathcal{N}(u;0,I)\,\exp\!\big(-\alpha\,\|G_\theta(s,u)-a\|\big),$$

by importance-sampled EM started from a Laplace (inverse-Jacobian) guess. The temperature α sets how close a decoded action must be to a to count; the effective sample size (ESS) of the EM tells us whether the fit is trustworthy (D3 gates on it). Everything is precomputed once (`tools/precompute_preimages.py`) and stored, one posterior per transition, next to a sidecar recording exactly which flow checkpoint produced it.

Sanity property worth remembering: if the flow perfectly modeled the behavior distribution, inverted latents would be exactly $\mathcal{N}(0,I)$ distributed, so $\|u^\star\|^2$ must look $\chi^2_{d_a}$. That is the D3 typicality test — a direct check that train-time latents and test-time latents come from the same distribution.

3.3 Stage C — successor measures over the latent-indexed family

A successor measure $M^\pi(s,a,X)$ is the discounted time a policy spends in the state set X after taking a at s. PSMFlows learns them for the whole family $\{\pi_{u'}\}$ at once, in factored form

$$M^{u\to u'}(s,X) \;\approx\; \psi(s,u',u)^\top \varphi(X),$$

read as: "take the action indexed by u now, then follow policy $\pi_{u'}$ forever." φ is a state basis (kept near-orthonormal by a regularizer), ψ carries all policy dependence. The training signal is the standard measure-matching TD loss (as PSM/FB): for a batch transition (s,a,s'),

- the *action slot* u is a sample from the stored preimage posterior $q_\alpha(\cdot \mid s,a)$ — the dataset action, expressed as a latent;
- the *policy index* u' is a fresh draw (half $\mathcal{N}(0,I)$, half permuted behavior preimages, clipped to the typical set);
- the bootstrap target is $\psi(s',u',u')$: at the next state, the continuation policy *is* the indexed policy, the same (Q,V) relationship as ordinary TD.

Note what is absent: no actor network, no argmax over actions, no decoded action anywhere in training. The flow only decodes at *acting* time. This is the sense in which every action the backup implicitly evaluates is data-supported.

4 Inference: reward in, policy out

Given rewards r on (a relabeled sample of) the dataset, two cheap steps:

1. **Task inference.** Because φ is near-orthonormal, $w = \mathbb{E}[\, r(s')\, \varphi(s')\,]$ is the least-squares projection of the reward onto the basis, and $\psi(s, u', u)^\top w \approx Q$-value of "do u, then follow $\pi_{u'}$" for this reward. Closed form, no training.

2. **Flow-GPI.** At each state draw K candidate action-latents and M policy indices, score every pair by $\psi(s, u', u)^\top w$ (minus a small ensemble- disagreement penalty), take the best candidate u, and decode it through the frozen flow: $a = G_\theta(s, u)$. This is generalized policy improvement over the latent family: at least as good as the best indexed policy in the sampled set, while every candidate action stays a flow decode.

5 The full algorithm

Algorithm 1 PSMFlows (as implemented on `feat/psm-integration`)

Input: reward-free dataset $\mathcal{D}$; later, a reward signal r
— Stage A: behavior flow (frozen afterwards) —
for each BC step **do**
  sample $(s, a) \sim \mathcal{D}$, $u_0 \sim \mathcal{N}(0, I)$, $t \sim U[0, 1]$
  regress $v_\theta(s, (1-t)u_0 + ta,\, t) \to (a - u_0)$; distill one-step decoder
end for
— Stage B: preimages (precomputed once) —
for each $(s_i, a_i) \in \mathcal{D}$ **do**
  $u_i^\star \leftarrow E_\theta(s_i, a_i)$ by backward ODE; Jacobian J_i by autodiff
  fit $q_\alpha(u \mid s_i, a_i)$ by IS-EM from the Laplace guess; record ESS
end for
validate: round-trip error, χ^2 typicality of $\{u_i^\star\}$, mean ESS (D3 gate)
— Stage C: successor-measure training —
for each TD step, batch $\{(s_i, a_i, s_i')\}$ **do**
  $u_i \sim q_\alpha(\cdot \mid s_i, a_i)$ ▷ dataset action as a latent
  $u_i' \leftarrow$ mix of $\mathcal{N}(0, I)$ and permuted $\{u_j\}$, clipped to $\pm u_{\text{clip}}$
  $M_{ij} \leftarrow \psi(s_i, u_i', u_i)^\top \varphi(s_j')$; $\bar{M}_{ij} \leftarrow \bar{\psi}(s_i', u_i', u_i')^\top \bar{\varphi}(s_j')$ ▷ targets
  descend the measure TD loss $+ \lambda_\perp \| \frac{1}{B} \sum_j \varphi\varphi^\top - I \|_F^2$; Polyak-update targets
end for
— Inference (zero-shot, per reward) —
$w \leftarrow \frac{1}{N} \sum_i r(s_i')\, \varphi(s_i')$, normalized
for each encountered state s **do**
  draw $u_{1:K}$, $u'_{1:M} \sim \mathcal{N}(0, I)$ clipped; $\hat{u} \leftarrow \arg\max_k \max_m \psi(s, u'_m, u_k)^\top w$
  act $a \leftarrow G_\theta(s, \hat{u})$ ▷ one-step decode; always data-like
end for

6 The diagnostics, and what each one certifies

	Tool	Question, in plain terms	Gate
D1	`validate_flow_fidelity`	does the flow reproduce the data's actions?	beat random control
D2	`eval_fixed_u_rollouts`	does a fixed u mean a distinct, repeatable policy?	ratio < 1
D3	`validate_flow_inversion`	can we trust the preimages?	ESS, typicality, roundtrip
D4	`latent_q_sanity`	with oracle rewards, does GPI actually control?	$\geq 50\%$ of FQL

Each stage only starts once the previous stage's gate passes, so a failure is localized: a bad D1 means the flow, a bad D3 means the inversion, a bad D4 with good D1–D3 means the representation or GPI — never a mystery spread across the whole pipeline.